

## Lab Assignment 6

### Experiment 6: AI-Based Code Completion — Working with Suggestions for Classes, Loops, Conditionals

|                       |               |
|-----------------------|---------------|
| <b>Name</b>           | M. Nikhil     |
| <b>Enrollment No.</b> | 2503A52693    |
| <b>Batch</b>          | 24            |
| <b>Course Code</b>    | 25CAI102PC205 |

#### Task Description #1 (AI-Based Code Completion for Loops)

Task: Use an AI code completion tool to generate a loop-based program.

Prompt:

“Generate Python code to print all even numbers between 1 and N using a loop.”

Expected Output:

- AI-generated loop logic.
- Identification of loop type used (for or while).
- Validation with sample inputs.

Code:

```
n = int(input("Enter N: "))
```

```
for i in range(2, n + 1, 2):
```

```
    print(i)
```

Output:

Enter N: 10

2

4

6

8

10

#### Task Description #2 (AI-Based Code Completion for Loop with Conditionals)

Task: Use an AI code completion tool to combine loops and conditionals.

Prompt:

“Generate Python code to count how many numbers in a list are even and odd.”

Expected Output:

- AI-generated code using loop and if condition.
- Correct count validation.
- Explanation of logic flow.

Code:

```
numbers = [12, 15, 18, 21, 24, 27]
```

```
even_count = 0
```

```
odd_count = 0
```

```
for num in numbers:
```

```
    if num % 2 == 0:
```

```
        even_count += 1
```

```
    else:
```

```
        odd_count += 1
```

```
print("Even numbers:", even_count)
```

```
print("Odd numbers:", odd_count)
```

Output:

Even numbers: 3

Odd numbers: 3

Task Description #3 (AI-Based Code Completion for Class Attributes Validation)

Task: Use an AI tool to complete a Python class that validates user input.

Prompt:

“Generate a Python class User that validates age and email using conditional statements.”

Expected Output:

- AI-generated class with validation logic.
- Verification of condition handling.
- Test cases for valid and invalid inputs.

Code:

```
class User:
```

```
    def __init__(self, age, email):
```

```
        self.age = age
```

```
        self.email = email
```

```
    def validate_age(self):
```

```
        if self.age < 0:
```

```
            return "Age cannot be negative."
```

```
        elif self.age < 18:
```

```
            return "User is a minor."
```

```
        else:
```

```
            return "User is an adult."
```

```
    def validate_email(self):
```

```
        if "@" in self.email and "." in self.email:
```

```
            return "Email is valid."
```

```
        else:
```

```
            return "Email is invalid."
```

```
#testing the User class
```

```
user1 = User(25, "user@example.com")
```

```
print(user1.validate_age())
```

```
print(user1.validate_email())
```

Output:

User is an adult.

Email is valid.

#### Task Description #4 (AI-Based Code Completion for Classes)

Task: Use an AI code completion tool to generate a Python class for managing student details.

Prompt:

“Generate a Python class Student with attributes (name, roll number, marks) and methods to calculate total and average marks.”

Expected Output:

- AI-generated class code.
- Verification of correctness and completeness of class structure.

Minor manual improvements (if needed) with justification.

Code:

class Student:

```
    def __init__(self, name, roll_number, marks):
```

```
        self.name = name
```

```
        self.roll_number = roll_number
```

```
        self.marks = marks
```

```
    def calculate_total(self):
```

```
        return sum(self.marks)
```

```
    def calculate_average(self):
```

```
        return sum(self.marks) / len(self.marks)
```

#testing the Student class

```
student1 = Student("Alice", 101, [85, 90, 78])
```

```
print("Total marks:", student1.calculate_total())
```

```
print("Average marks:", student1.calculate_average())
```

Output:

Total marks: 253

Average marks: 84.33333333333333

### Task Description 5 (AI-Assisted Code Completion Review)

Task: Use an AI tool to generate a complete Python program using classes, loops, and conditionals together.

Prompt:

“Generate a Python program for a simple bank account system using class, loops, and conditional statements.”

Expected Output:

- Complete AI-generated program.
- Identification of strengths and limitations of AI suggestions.

Reflection on how AI assisted coding productivity.

Code:

class BankAccount:

```
def __init__(self, account_number, account_holder, balance=0):
```

```
    self.account_number = account_number
```

```
    self.account_holder = account_holder
```

```
    self.balance = balance
```

```
def deposit(self, amount):
```

```
    if amount > 0:
```

```
        self.balance += amount
```

```
        print(f"Deposited: {amount}. New balance: {self.balance}")
```

```
    else:
```

```
        print("Deposit amount must be positive.")
```

```
def withdraw(self, amount):
```

```
    if amount > 0:
```

```
        if amount <= self.balance:
```

```
            self.balance -= amount
```

```
        print(f"Withdrew: {amount}. New balance: {self.balance}")
    else:
        print("Insufficient funds.")
    else:
        print("Withdrawal amount must be positive.")

    def display_balance(self):
        print(f"Account Holder: {self.account_holder}, Balance: {self.balance}")

#testing the BankAccount class
account1 = BankAccount("123456", "John Doe", 1000)
print("Initial balance:")
account1.display_balance()
account1.deposit(500)
account1.withdraw(200)
```

Output:

Initial balance:

Account Holder: John Doe, Balance: 1000

Deposited: 500. New balance: 1500

Withdrew: 200. New balance: 1300